Training Mixture of Experts

Ronit Anandani, Dev Singh, Ryan To, Kaushik Varadharajan, Yanni Zhuang

{ronita2,dsingh14,rto4,kv22,yanniz3}@illinois.edu

The Parameter Scaling Problem

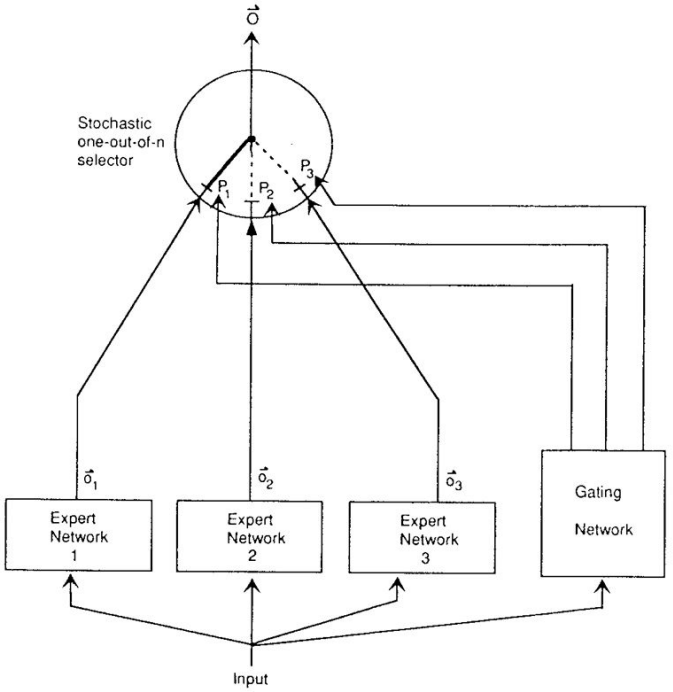
- Generally more parameters → better models
- This leads to quadratic blowup in training time
- Conditional compute was proposed by various papers
  - Parts of the network are active or inactive
  - Compute remains the same, parameter count can balloon
  - Determined by a gating mechanism
    - binary, sparse, stochastic or deterministic

Failings of Conditional Computation

- Benefits of conditional computation recognized, but not realized
  - We can reduce active parameter count at inference time
  - Minor improvements made in previous work, but none massive
- GPUs are faster at arithmetic than branching
- Large batch sizes are critical for performance
- Network bandwidth can be a bottleneck
- Specialized loss required for enforcing sparsity and load balancing

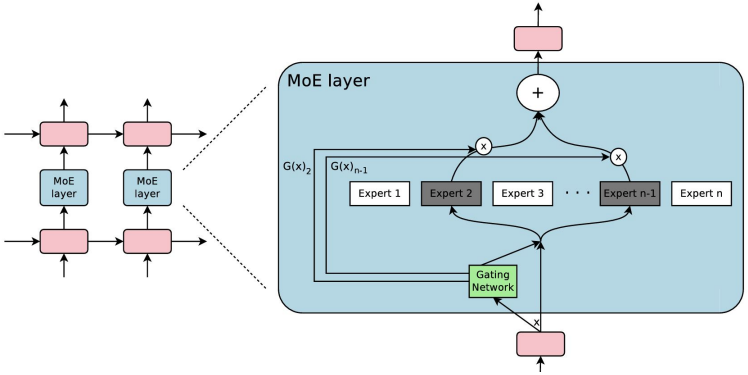
Classical Mixture of Experts

- Jacobs et al. (1991) proposed Mixture of Experts (MoE)
  - The MoE is the entire model
  - Train expert networks and gating mechanism
- Eigen et al. (2013) proposed using MoE as a component of a model



Sparsely-Gated Mixture of Experts

- Proposed by Shazeer et al. (2017)
- MoE called once for each position in text
  - Selects a different combination of experts for each position in the text
- *n* expert networks (FFNs)
- Learns a gating network G(x)
  - Outputs *n-dim* vector
  - Output is sparse, when output is 0 you do not need to evaluate a given expert.
- MoE output: $y = \Sigma\, G(x)_i \cdot E_i(x)$
- Can be hierarchical



Expert 381	Expert 752	Expert 2004
... with **researchers** , ...	... plays **a** core ...	... with **rapidly** growing ...
... to **innovation** .	... plays **a** critical ...	... under **static** conditions ...
... tics **researchers** .	... provides **a** legislative ...	... to **swift** ly ...
... the **generation** of ...	... play **a** leading ...	... to **dras** tically ...
... technology **innovations** is ...	... assume **a** leadership ...	... the **rapid** and ...
... technological **innovations** , ...	... plays **a** central ...	... the **fast** est ...
... support **innovation** throughout ...	... taken **a** leading ...	... the **Quick** Method ...
... role **innovation** will ...	... established **a** reconciliation ...	... rec **urrent**) ...
... research **scienti** st ...	... played **a** vital ...	... provides **quick** access ...
... promoting **innovation** where ...	... have **a** central ...	... of **volatile** organic ...
...	...	...

Sparse Gating Network

- A non-sparse gating network: $G_\sigma(x) = Softmax(x \cdot W_g)$
- Adding Gaussian noise helps load balance
- Keep only top-k
  - The rest get turned to -∞ (0 after softmax)
- Softmax
- Training is done by back-propagation

$$G(x) = Softmax(KeepTopK(H(x), k))$$

$$H(x)_i = (x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i)$$

$$KeepTopK(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases}$$

Hierarchical MoE

- If the number of experts is large, reducing the branching factor is possible
- Layer the MoE, with the experts each being their own Sparsely-Gated MoE

$$y_H = \sum_{i=1}^{a} \sum_{j=1}^{b} G_{primary}(x)_i \cdot G_i(x)_j \cdot E_{i,j}(x)$$

$$Importance_H(X)_{i,j} = \sum_{x \in X} G_{primary}(x)_i \cdot G_i(x)_j$$

$$Load_H(X)_{i,j} = \frac{Load_{primary}(X)_i \cdot Load_i(X^{(i)})_j}{|X^{(i)}|}$$

Issues with naive approach

- Model may collapse to a handful of experts
  - As training goes on, one expert will become very strong
- Batch size per expert becomes too low
  - Experts no longer get the benefits of high batch size

Importance Loss

- Prevents the natural state of *G* converging to a state where select experts have high weight
  - This overweighting can cause resource utilization imbalance
- This loss is added to model loss, incentivizing experts to have equal importance
  - Adding allows the gradients from this loss to flow back through the network
- "Importance of an expert" is the sum of gating values for said expert
- $L_{importance}(X) = w_{importance} \cdot CoeffVariation(Importance(X))^2$
  - $w_{importance}$ is hand chosen
- $L_{total} = L_{model} + L_{importance}$

Load Loss

- Experts may still get different numbers of examples
  - Same amount of weights *is not* same number of examples
  - We want a similar number of examples on all experts, regardless of weight size
  - This imbalance can overwhelm a single server or GPU
- Number of examples is non-differentiable
  - Discrete variables must be converted to some continuous function
  - Thus, the paper defines a smooth estimator which backprop can be done on

$$P(x,i) = \Phi\left(\frac{(x \cdot W_g)_i - kth\_excluding(H(x),k,i)}{Softplus((x \cdot W_{noise})_i)}\right)$$

P(x, i): The probability that for input *x* (of batch X), that expert *i* is activated, given a new random choice of noise term.

(Φ is the CDF of the normal distribution)

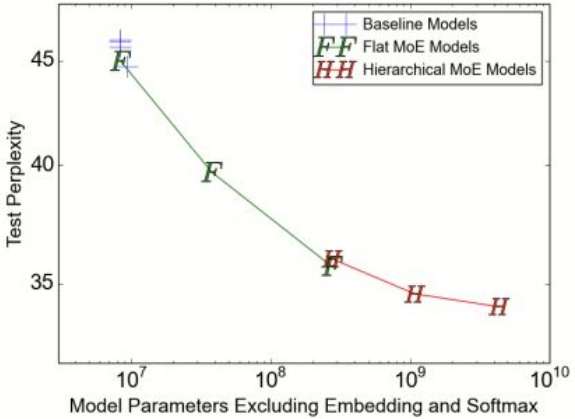
$$Load(X)_i = \sum_{x \in X} P(x,i)$$

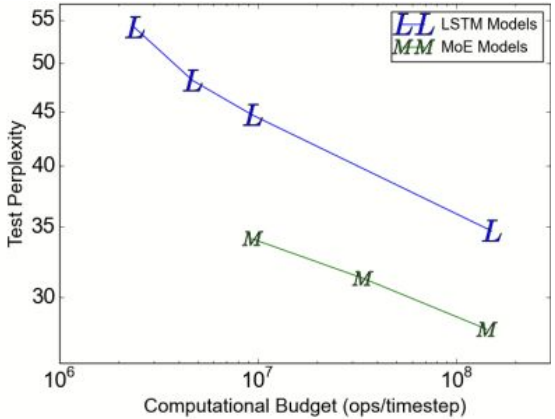
$$L_{load}(X) = w_{load} \cdot CV(Load(X))^2$$

Load loss in detail

Results - Word Language Modeling

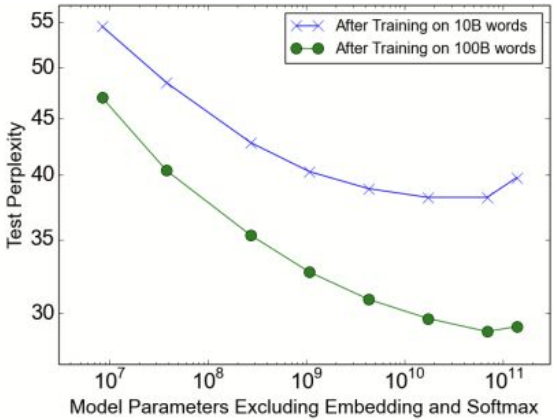
- Dataset has ~829M words (~793K unique), contains shuffled unique sentences from news articles
- Tested models with 4, 32, 256 flat MoEs, and 256, 1024, and 4096 hierarchical MoEs
  - 4 experts active for each model
- Largest model achieves 24% lower perplexity on test set, the more experts the better





Results - 100 Billion Word Google News

- Dataset has 100 billion words, shuffled unique sentences from Google's internal news corpus
- Tested 10^7, 10^8, 10^9, 10^{10}, 10^{11} parameter models
- Perplexity decreased until largest model
  - Paper attributes this to model sparsity becoming too high
  - Does not fully explain *why* this occurs



Model	Test Perplexity .1 epochs	Test Perplexity 1 epoch	ops/timestep (millions)	#Params excluding embed. & softmax (millions)	Total #Params (billions)	TFLOPS per GPU (observed)
Kneser-Ney 5-gram	67.1	45.3	0.00001		76.0	
4xLSTM-512	54.5	47.0	8.4	8.4	0.1	**1.23**
MoE-32	48.5	40.4	8.4	37.8	0.1	0.83
MoE-256-h	42.8	35.3	8.4	272.9	0.4	1.11
MoE-1024-h	40.3	32.7	8.5	1079.0	1.2	1.14
MoE-4096-h	38.9	30.9	8.6	4303.4	4.4	1.07
MoE-16384-h	**38.2**	29.7	8.8	17201.0	17.3	0.96
MoE-65536-h	**38.2**	**28.9**	9.2	68791.0	68.9	0.72
MoE-131072-h	39.8	29.2	9.7	137577.6	137.7	0.30

Results - Machine Translation

- 36M English → French sentence translation pairs, 5M English → German translation pairs
- **Previous SOTA perplexity: 2.79** (GNMT, Wu et al, 2016)
- **MoE with 2048 experts perplexity: 2.69 when trained for 3 days**
  - Perplexity drops to 2.63 when trained for 6 days
- BLEU: used to measure translation quality, 0 → 100

Model	Test Perplexity	Test BLEU	ops/timenstep	Total #Parameters	Training Time
MoE with 2048 Experts	2.69	40.35	85M	8.7B	3 days/64 k40s
MoE with 2048 Experts (longer training)	**2.63**	**40.56**	85M	8.7B	6 days/64 k40s
GNMT (Wu et al., 2016)	2.79	39.22	214M	278M	6 days/96 k80s
GNMT+RL (Wu et al., 2016)	2.96	39.92	214M	278M	6 days/96 k80s
PBMT (Durrani et al., 2014)		37.0			
LSTM (6-layer) (Luong et al., 2015b)		31.5			
LSTM (6-layer+PosUnk) (Luong et al., 2015b)		33.1			
DeepAtt (Zhou et al., 2016)		37.7			
DeepAtt+PosUnk (Zhou et al., 2016)		39.2			

$w_{importance}$	w_{load}	Test Perplexity	CV(Importance(X))	CV(Load(X))	max(Load(X)) / mean(Load(X))
0.0	0.0	39.8	3.04	3.01	17.80
0.2	0.0	**35.6**	0.06	0.17	1.47
0.0	0.2	35.7	0.22	0.04	1.15
0.1	0.1	**35.6**	0.06	0.05	1.14
0.01	0.01	35.7	0.48	0.11	1.37
1.0	1.0	35.7	0.03	0.02	**1.07**

Contributions of Importance and Load loss

(Lower perplexity is desired)

Strengths

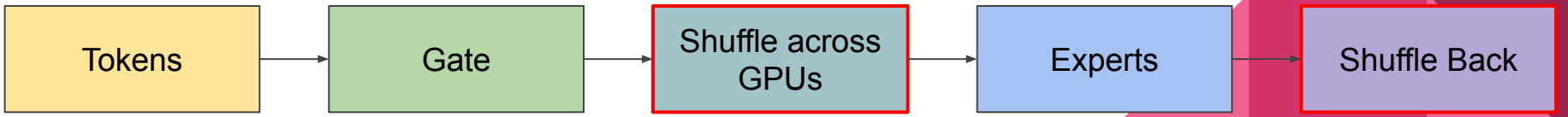
- Realizes theoretical scalability
  - Greater than 1000x improvement in model scalability
- Strong empirical results
  - Beats SOTA at lower computational cost
- Load balancing analysis is strong
  - Addresses failure mode where one expert is over-trained
  - Introduce and rigorously analyze
- Expert specialization analysis increases interpretability and builds intuition

Weaknesses

- Doesn’t use Transformers
  - “Attention is all you need” paper was published later that year
  - Faces the inherent sequential limitations of LSTM - uses convolutions to work around this
- Hyperparameter sensitivity not well motivated
- Infrastructure brittleness
  - Authors admit that "peculiarities in our infrastructure" forced them to use specific gating functions to maintain speed
  - Suggests the system is brittle and sensitive to hardware changes
- No analysis of inference-time performance
  - Due to small batches at inference, batch-wise masking efficiency may be lost
-

MoE solves capacity, but breaks training efficiency

- Tokens dynamically routed → frequent All-to-All communication
- Experts distributed across GPUs → constant reshuffling
- Small per-expert batches → poor utilization
- GPUs wait on network → communication dominates compute
- Training becomes communication-bound, not compute-bound



MoE Communication Bottleneck

- Communication overhead typically contributes over 50% to the overall training

Testbeds/Breakdown		Communication				Computation			
		AlltoAll	AllReduce	AllGather	ReduceScatter	Experts	Routing	Order	Attention
A	GPT2-Forward	6.9(31.16%)	0(0%)	4.6(20.83%)	5.4(24.46%)	3.1(14.04%)	0.1(0.45%)	0.3(1.36%)	1.7(7.7%)
	GPT2-Backward	6.9(21.27%)	5.26(16.26%)	4.6(14.22%)	5.4(16.7%)	6.1(18.86%)	0.1(0.31%)	0.4(1.24%)	3.6(11.13%)
	Mixtral-Forward	19.5(29.8%)	0(0%)	12.3(18.73%)	13.7(20.86%)	15.6(23.76%)	0.1(0.15%)	0.3(0.46%)	4.1(6.24%)
	Mixtral-Backward	19.6(17.45%)	26.45(23.59%)	12.3(10.97%)	13.7(12.22%)	31.8(28.36%)	0.1(0.09%)	0.5(0.45%)	7.7(6.87%)
B	GPT2-Forward	11.2(20.7%)	0(0.0%)	15.5(28.7%)	15.7(29.1%)	6.7(12.4%)	0.1(0.2%)	0.3(0.6%)	4.5(8.3%)
	GPT2-Backward	11.2(15.7%)	7.3(10.3%)	15.5(21.8%)	15.2(21.3%)	13(18.3%)	0.1(0.1%)	0.3(0.4%)	8.6(12.1%)
	Mixtral-Forward	28.3(15.9%)	0.0(0.0%)	39.6(22.3%)	40.8(23.0%)	58.5(33.0%)	0.1(0.1%)	0.7(0.4%)	9.5(5.4%)
	Mixtral-Backward	30.8(10.8%)	32.1(11.3%)	40.1(14.1%)	41.8(14.7%)	119.7(42.1%)	0.2(0.1%)	1.2(0.4%)	18.1(6.4%)

DIVIDER FOR NEW PAPER

Paradigms of parallelism for MoE models

General distributed training:

- Data Parallelism
- Model Parallelism
  - Tensor Parallelism
  - Pipeline Parallelism (not pictured)

Specific to MoE:

- Expert Parallelism
- Expert-Sharding Parallelism

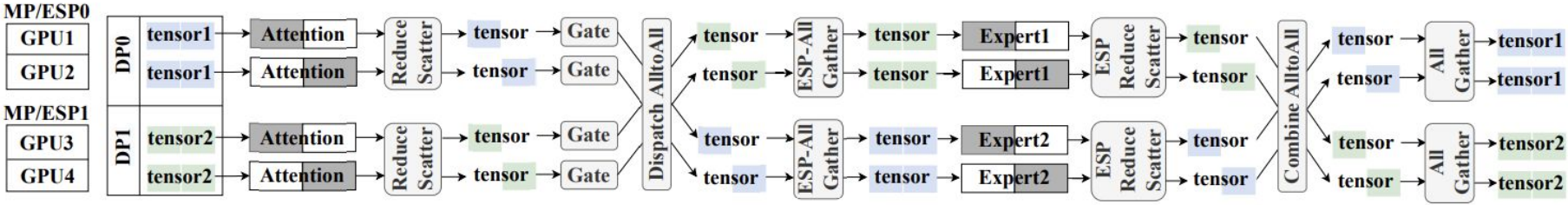
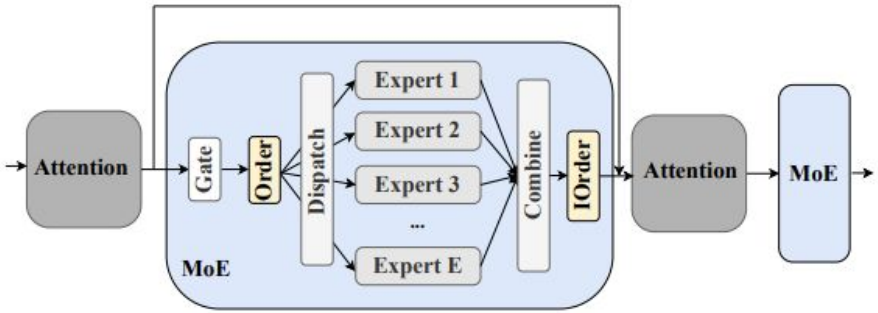


Figure 2. An example of $N_{DP} = N_{MP} = N_{EP} = N_{ESP} = 2$. The attention is partitioned into two parts across MP groups, and the two experts are distributed to the two EP groups (GPU1 and GPU3, as well as GPU2 and GPU4) in EP, and each expert is further partitioned into two shards across the ESP group. The blue and green rectangles indicate the data tensors.

An MoE forward pass, in practice

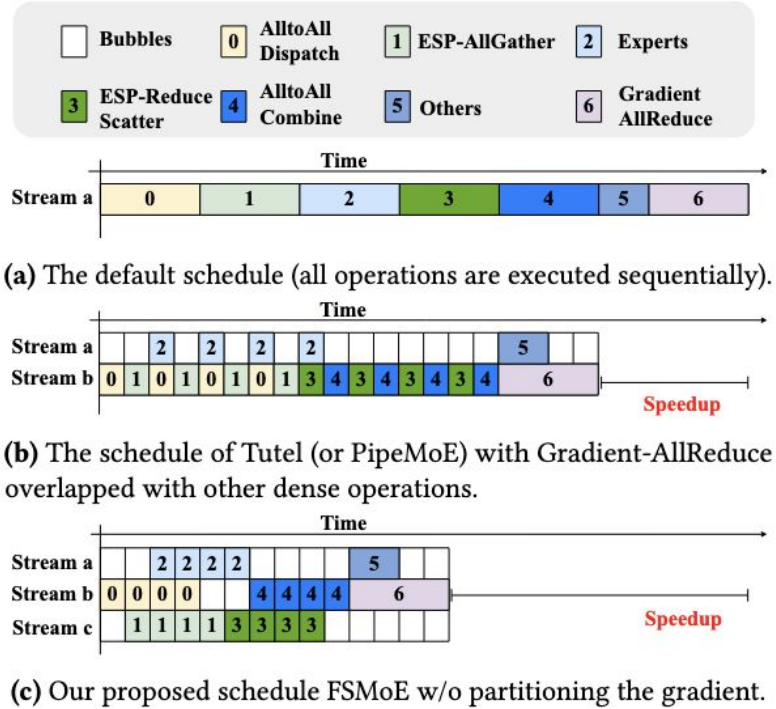
- After the gating function, the **Order** function transforms the input in preparation for dispatching
  - The **IOrder** does the opposite after the experts, in preparation for the next attention
- **Dispatch** and **Combine** are GPU communication steps to move input data to the correct experts
- The **Experts** require intra-node communication if they are sharded across GPUs



Resource Overlap in FSMoE (Pan et al.)

- GPU resources used during the MoE layer:
  - **Compute** (experts)
  - **Intra-node** communication (ESP)
  - **Inter-node** communication (EP)
- Uses of these resources can be *overlapped*, making **pipelining** an effective optimization
  - This requires the input to be processed in chunks; the number of chunks (*pipeline degree*) impacts pipeline effectiveness

FSMoE

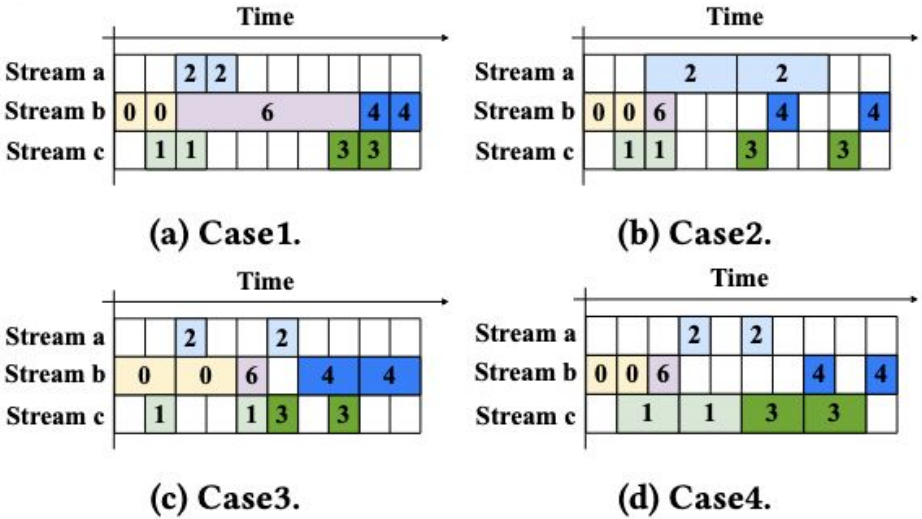


(a) The default schedule (all operations are executed sequentially).

(b) The schedule of Tutel (or PipeMoE) with Gradient-AllReduce overlapped with other dense operations.

(c) Our proposed schedule FSMoE w/o partitioning the gradient.

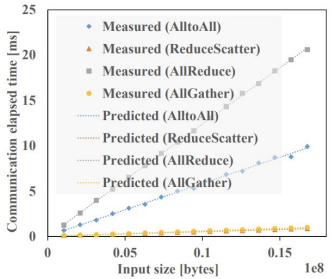
Note: backward pass, pipeline degree r=4



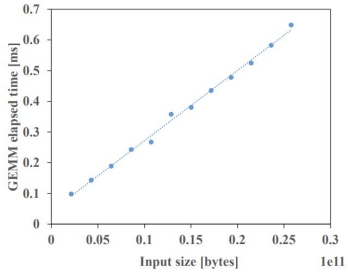
Four bottlenecks possible in the MoE pipeline dependent on pipeline degree.

Optimizing the pipeline degree

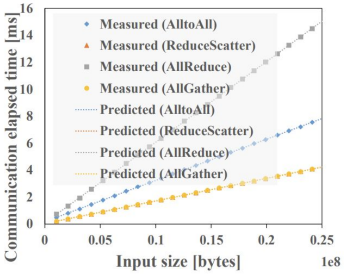
- Each layer's runtime is linear w.r.t. the chunk size
  - The linear coefficients are found by microbenchmarking
- The MoE pipeline's runtime can be expressed in terms of these coefficients and mathematically optimized



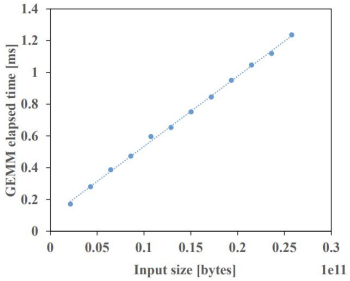
(a) Communication (A6000).



(b) GEMM (A6000).



(c) Communication (2080Ti).



(d) GEMM (2080Ti).

Algorithm 1 FindOptimalPipelineDegree

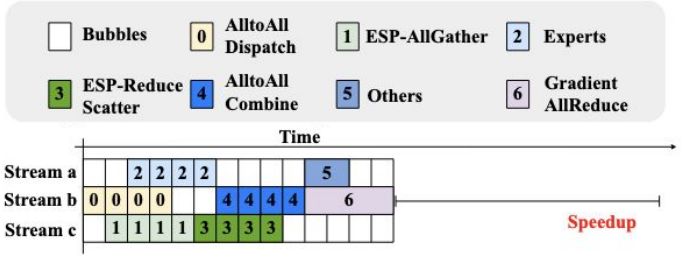
Input: $\alpha_{a2a}, \beta_{a2a}, n_{a2a}, \alpha_{ag}, \beta_{ag}, n_{ag}, \alpha_{rs}, \beta_{rs}, n_{rs}, \alpha_{exp}, \beta_{exp}, n_{exp}, t_{gar}$
Output: r and t^{moe}

1: $r1, t1 = solve(f_1)$ ▷ Solve with SLSQP
2: $r2, t2 = solve(f_2)$
3: $r3, t3 = solve(f_3)$
4: $r4, t4 = solve(f_4)$
5: candidate_mins = $[t1, t2, t3, t4]$
6: candidates = $[r1, r2, r3, r4]$
7: r = candidates[argmin(candidate_mins)]
8: $t^{moe} = min$(candidate_mins)
9: return r and t^{moe}.

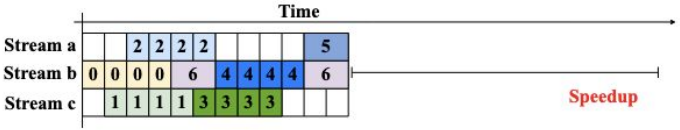
$$t_{a2a,r} = \alpha_{a2a} + \frac{n_{a2a}}{r} \cdot \beta_{a2a},$$
$$t_{ag,r} = \alpha_{ag} + \frac{n_{ag}}{r} \cdot \beta_{ag},$$
$$t_{rs,r} = \alpha_{rs} + \frac{n_{rs}}{r} \cdot \beta_{rs},$$
$$t_{exp,r} = \alpha_{exp} + \frac{n_{exp}}{r} \cdot \beta_{exp},$$

Adaptive Gradient Partitioning

- See **0-4-6**; we can't overlap Gradient-AllReduce with AlltoAll because they're both inter-node.
- Solution: split gradients into chunks and strategically assign them to overlap with different parts of the MoE layer that don't use inter-node communication.
  - The optimal partition is again modeled as an optimization problem and solved



(c) Our proposed schedule FSMoE w/o partitioning the gradient.



(d) Our proposed schedule FSMoE w/ partitioning the gradient.

As the optimization will be conducted only once before the training, we do not need to care too much about the time cost. Therefore, we simply adopt the differential evolution algorithm [35] when we solve the above optimization problem.

Results: Speedup in Training Time

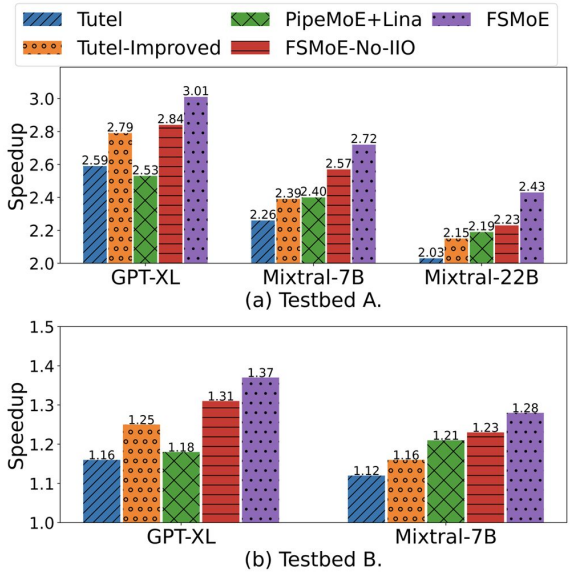
Performance on Configured Layers

Schedule	Speedup	
	Testbed-A	Testbed-B
Tutel	1.00×	1.00×
Tutel-Improved	1.09×	1.08×
FSMoE-No-IIO	1.12×	1.16×
FSMoE	1.18×	1.22×

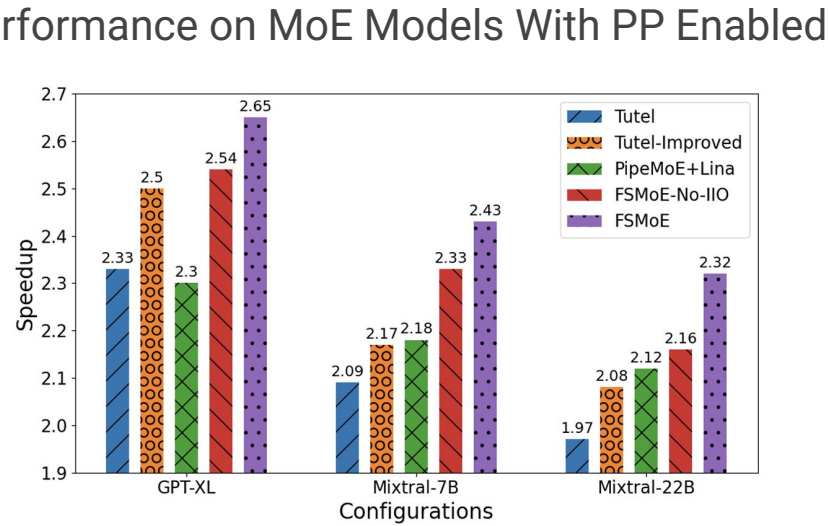
Testbed A: Dual Intel Xeon Plat. 8358 CPU, 8x Nvidia RTX A6000, 512GB DDR4, 112.5GB/s NVlink

Testbed B: Dual Intel Xeon Gold 6230, 4x Nvidia RTX2080Ti, 512GB DDR4, No NVLink

Performance on MoE Models

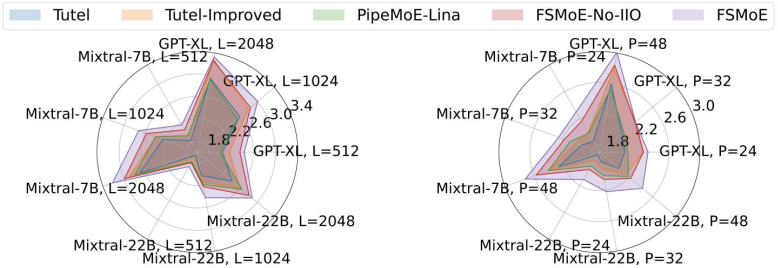


Results: Speedup in Training Time



Results: Speedup in Training Time

Performance on MoE Models with Varied L and P



(a) Speedups with varied L. (b) Speedups with varied P.

Performance with Varied Gating Functions

Gating	DeepSpeed-MoE	FSMoE
Gshard [22]	968.1 ± 1.4	707.7 ± 1.6(1.37×)
X-MoE [6]	1064.0 ± 1.5	746.9 ± 2.8(1.42×)
Sigmoid [23]	986.6 ± 1.4	721.0 ± 1.8(1.37×)
EC [51]	909.9 ± 1.8	685.5 ± 1.5(1.33×)

Strengths and Weaknesses

Strengths:

- Modular abstraction allows all parallelism paradigms to be exploited
- Overlapping of intra-node and inter-node communication allows for more effective resource usage
- Various scheduling cases and adaptable chunking allow improves performance across diverse workloads

Weaknesses:

- The optimal schedule is most effective when MP and ESP groups align with the number of GPUs; performance when groups are uneven across nodes is unclear.
- Efficiency of pipelining relies on fast intranode communication

Future Directions

- Develop scheduling strategies for when MP/ESP groups do not align with node boundaries
- Instead of just profiling once before training, explore dynamic adaptations to handle runtime variations
- Instead of pinning experts to specific GPUs, experts can migrate to underutilised GPUs